

Ejercicio 6: El Problema del Cambio de Monedas (Programación Dinámica)

Enunciado

Se trata de pagar una cantidad N a un cliente usando el menor número posible de monedas. Suponemos que en nuestro sistema de monedas hay n tipos diferentes, $\{1, 2, \dots, n\}$, de forma que la moneda i vale d_i unidades ($d_i > 0$), y que hay un número ilimitado de monedas de todos los tipos. Diseñar un algoritmo de programación dinámica que funcione para cualquier sistema de monedas.

1. Estructura de Programación Dinámica

Definición de la Función Óptima

Definimos un vector DP de tamaño $N + 1$, donde:

- $DP[i]$ es el **mínimo número de monedas** necesarias para sumar exactamente la cantidad i .

Relación de Recurrencia

Para conseguir la cantidad i , podemos probar a usar como última moneda cualquiera de las denominaciones disponibles d_j que no superen el valor i . El coste será 1 (por usar la moneda d_j) más el coste óptimo de pagar el resto ($i - d_j$):

$$DP[i] = \min_{1 \leq j \leq n, d_j \leq i} \{DP[i - d_j] + 1\}$$

Casos Base e Inicialización

- Caso Base:** $DP[0] = 0$ (se necesitan 0 monedas para pagar 0 unidades).
- Inicialización:** $DP[i] = \infty$ para todo $i > 0$ (representa que el valor es inalcanzable de momento).

2. Algoritmo y Complejidad

Pseudocódigo

Entrada: $D[1..n]$ (valores de las monedas), N (cantidad a devolver)

$DP[0] = 0$

Para i desde 1 hasta N :

$DP[i] = \text{infinito}$

// Array para la reconstrucción de las monedas elegidas

```

moneda_usada[0..N] = -1

Para i desde 1 hasta N:
    Para j desde 1 hasta n:
        Si (D[j] <= i):
            Si (DP[i - D[j]] + 1 < DP[i]):
                DP[i] = DP[i - D[j]] + 1
                moneda_usada[i] = D[j] // Guardamos la moneda elegida

Si DP[N] == infinito:
    Retornar "No es posible dar el cambio exacto"
Sino:
    // Reconstrucción del cambio
    Monedas_Elegidas = []
    aux = N
    Mientras (aux > 0):
        Añadir moneda_usada[aux] a Monedas_Elegidas
        aux = aux - moneda_usada[aux]

    Retornar DP[N], Monedas_Elegidas

```

Análisis de Coste

- **Temporal:** $O(N \cdot n)$. Tenemos un bucle exterior que va hasta la cantidad N y un bucle interior que recorre los n tipos de monedas. En cada celda hacemos operaciones de coste constante $O(1)$.
- **Espacial:** $O(N)$ para almacenar los arrays de estado `DP` y de reconstrucción `moneda_usada`.